AutoPatch 학습 주석본

한 줄 요약

LLM은 학습 마감일(knowledge cutoff) 이후에 공개된 CVE를 모르기 때문에, 이미 알려진 취약 패턴을 그대로 담은 코드를 만들어 낸다. AutoPatch는 최신 CVE 75개(코드 525개)를 RAG DB에 넣어 두고, LLM이 만든 코드와 의미·데이터 흐름이 가장 닮은 CVE를 찾은 뒤 그 CVE의 원인 분석과 패치 사례를 예시로 붙여 LLM에게 검증과 패치를 시킨다. CVE 매칭 91.8%, 검증 F1 90.3%, 패치 정확도 94.1%, 미세조정(fine-tuning)보다 비용이 최대 52배 적다. 출처: Minjae Seo, Wonwoo Choi, Seungwon Shin, Myoungsung You, arXiv 2505.04195v2 (2025). 유명성 교수님 교신저자.

읽는 순서

1) 초록과 그림 1 (IDE 플러그인으로 어떻게 끼어드는가) 2) 2장 그림 2 동기 예시 (cutoff 전후 CVE 차이) 3) 3장은 그림 3을 옆에 두고 세 에이전트(Similarity Analyzer, Vulnerability Verifier, Code Patcher) 순서로 4) 식 (2) 통합 점수와 순위 손실은 개념만 5) 5장은 표 IV·V와 그림 7(비용)만 먼저 6) 6장 한계 7) 부록 C 그림 9에 실제 프롬프트 전체가 있다. 프롬프트 구조가 궁금하면 여기부터 봐도 된다. 적용 범위 주의: 논문 대상은 IDE에서 LLM이 생성한 코드다. 펌웨어 백포트나 카메라 SDK 코드는 논문이 다루지 않았다.

표시 규칙

노랑: 핵심 주장, 결과 수치

초록: 전문용어. 오른쪽에 풀이

분홍: 한계, 가정, 위협 모델

①②… 빨간 번호 = 오른쪽 여백의 같은 번호 설명. → 표시 = EN683·한화비전과 연결되는 지점.

이 판에는 원문 쪽이 없다. 원문을 옆에 띄워 놓고 쪽 번호와 메모 번호로 따라 읽는다. 형광펜 문구는 원문에서 그대로 옮겨 적었다. 원문: https://arxiv.org/abs/2505.04195

용어집 (등장 순서)

LLM(대형 언어 모델): 대량의 글과 코드를 학습해 다음 단어(토큰)를 예측하는 모델. ChatGPT, Code Llama, DeepSeek 등. 코드 생성에도 쓴다.

knowledge cutoff(학습 마감일): 학습 데이터를 모은 마지막 시점. 그 뒤에 나온 CVE와 패치는 모델이 모른다. 이 논문의 문제의식 전체가 여기서 나온다.

CVE: 공개된 개별 취약점에 붙는 고유 번호(예: CVE-2025-21671). MITRE가 관리한다. 특정 소프트웨어의 특정 버그 하나를 가리킨다.

fine-tuning(미세조정): 이미 학습된 모델을 새 데이터로 추가 학습해 가중치를 바꾸는 것. GPU 시간이 많이 들어서 CVE가 나올 때마다 하기 어렵다.

CWE: 취약점의 종류(유형) 분류. 예: CWE-416 Use After Free, CWE-787 버퍼 넘침. CVE는 사례, CWE는 범주다. 기존 연구는 교과서식 CWE 예제만 써서 실제 코드에 약하다.

multi-agent: 역할이 다른 LLM 호출(에이전트) 여러 개를 이어 붙여 일을 나눠 맡기는 구조. 여기서는 유사도 분석, 검증, 패치 세 에이전트.

RAG(검색 증강 생성): 질문할 때 외부 DB에서 관련 자료를 찾아 프롬프트에 붙여 주는 방식. 모델 가중치를 바꾸지 않고 새 지식을 넣을 수 있다.

CoT(사고 사슬): LLM에게 답만 내지 말고 단계별 추론을 쓰게 하는 프롬프트 기법. 복잡한 판단의 정확도가 오른다.

semantic similarity(의미 유사도): 코드가 무엇을 하려는지(설명문, 키워드)가 CVE 코드와 얼마나 닮았는지 점수로 낸 것.

data flow analysis(데이터 흐름 분석): 변수 값이 어디서 만들어져 어느 함수로 흘러가는지 추적하는 정적 분석. 이름이 달라도 구조가 같으면 잡아낸다.

pairwise ranking loss: 정답 CVE 점수가 오답 CVE 점수보다 일정 간격 이상 높도록 학습시키는 손실 함수. 분류가 아니라 순위를 맞추는 학습이다.

one-shot 예시: 프롬프트에 풀이 예제를 하나 붙여 주는 것. 여기서는 찾아낸 CVE의 실제 취약 코드와 원인 분석이 예제가 된다.

표 I: 대표 코드 생성 모델들의 학습 마감일. GPT-4o, o3-mini가 2023년 10월이다. 그 뒤 CVE는 모른다는 근거.

inference(추론): 학습이 끝난 모델을 실제로 호출해 답을 받는 단계. RAG 검색은 이 시점에 일어난다.

zram: RAM 일부를 압축 블록 장치로 쓰는 리눅스 커널 기능. 메모리가 작은 임베디드 기기에서 스왑 대신 자주 쓴다.

dangling pointer(허상 포인터): 메모리를 해제한 뒤에도 그 주소를 계속 가리키는 포인터. NULL로 지워 두지 않으면 나중에 다시 쓰일 수 있다.

Use After Free(UAF, CWE-416): 해제된 메모리를 다시 접근하는 버그. 공격자가 그 자리에 원하는 데이터를 넣으면 코드 실행까지 이어진다.

그림 2 읽는 법: 위는 cutoff 이후 CVE(취약하게 생성), 아래는 cutoff 이전 CVE(안전하게 생성). 같은 모델, 같은 방식의 프롬프트로 비교한 것이다.

그림 3 읽는 법: 왼쪽에서 개발자 요청이 들어오고, 생성 코드가 Similarity Analyzer → Verifier → Patcher를 거친다. 오른쪽 RAG DB가 각 단계에 CVE 정보를 공급한다.

CVE-2022-0185: 리눅스 파일시스템 마운트 파라미터 처리에서 길이 계산 정수 오류로 힙 버퍼가 넘치는 취약점. 컨테이너 탈출에 실제 악용되었다.

배포 시나리오: 개발자가 Copilot, Cursor 같은 LLM IDE를 쓰고, AutoPatch가 생성 코드를 IDE에 넣기 전에 가로채 검사한다.

두 능력: (i) 의미 분석은 무엇을 하는 코드인지 비교, (ii) 데이터 흐름 분석은 변수·함수 이름을 지우고 역할과 흐름만 비교. 둘을 합쳐야 구조가 다르지만 같은 버그인 경우도 잡는다.

Unified Similarity Model: 키워드, 설명문, 변수, 함수 네 가지 유사도에 가중치를 곱해 더한 점수 모델. 가중치를 학습으로 정한다.

키워드 사전: Stack Overflow 상위 태그 1만 개를 키워드 후보로 쓴다. 코드 설명에서 이 태그에 해당하는 단어만 뽑는다.

Jaccard 유사도: 두 집합의 교집합 크기 ÷ 합집합 크기. 키워드가 많이 겹칠수록 1에 가깝다.

rapidfuzz: 철자가 조금 달라도 비슷한 문자열을 찾는 퍼지 매칭 라이브러리. 유사도 80% 이상이면 같은 키워드로 친다(예: alloc과 allocate).

embedding(임베딩): 문장이나 코드 설명을 수백~수천 차원 숫자 벡터로 바꾼 것. 뜻이 비슷하면 벡터 방향도 비슷해진다.

코사인 유사도: 두 벡터 사이 각도의 코사인. 방향이 같으면 1, 직각이면 0. 길이가 아닌 방향(의미)만 비교한다.

DFG(데이터 흐름 그래프): 노드는 변수·함수, 화살표는 값이 흘러가는 방향. 그림 4가 zram 코드에서 뽑은 DFG다.

CodeQL: GitHub의 정적 분석 도구. 소스 코드를 DB로 바꾼 뒤 SQL 비슷한 질의로 데이터 흐름을 찾는다. → 펌웨어 C 코드 감사에도 그대로 쓸 수 있는 도구다.

그림 5 읽는 법: LLM에게 DFG 정보를 주고 각 변수·함수를 이름 없이 설명하게 한 결과. 예: vzalloc → 0으로 초기화한 연속 가상 메모리 할당.

margin(여유폭) m: 정답 점수가 오답 점수보다 최소 m만큼은 높아야 손실이 0이 된다. 하이퍼파라미터는 학습 전에 사람이 정하는 값.

Adam: 가장 널리 쓰는 경사하강 최적화 알고리즘. 손실이 줄어드는 방향으로 가중치를 조금씩 조정한다.

APR(Automated Program Repair, 자동 프로그램 수리): 버그를 자동으로 고치는 연구 분야. 패치 → 검증 → 다시 패치 루프가 표준 구조다.

그림 6 읽는 법: 점선 상자와 파란 칸이 검증용, 초록 칸이 패치용 프롬프트 부품. system → one-shot 예시 → user 순서로 이어 붙인다.

role-based 프롬프트: system(규칙과 역할), user(질문), assistant(답) 역할로 나눠 쓰는 대화형 LLM 입력 형식.

in-context learning: 가중치를 바꾸지 않고 프롬프트 안의 예시만 보고 따라 하게 하는 것. one-shot 예시가 그 역할을 한다.

one-hop calling function: 대상 함수가 직접 호출하는 함수(한 단계 거리). 그 정의까지 같이 줘야 LLM이 동작을 이해한다.

Code Patcher 역할 세 가지: 패치 예시 구성, 패치 프롬프트 생성, 결과를 다시 Verifier로 넘겨 재검증.

표 II 읽는 법: trivial(겉모양만 바꿈: 공백, 주석, 도달 불가 코드)과 non-trivial(구조 변경: 이름 변경, 위험 함수 안전 사용)로 나눈 데이터 증강.

LangChain: LLM 호출, 프롬프트 템플릿, DB 검색을 묶어 에이전트를 만드는 파이썬 프레임워크.

vector DB(벡터 검색): 임베딩 벡터를 저장해 두고 가장 가까운 벡터를 빠르게 찾는 DB. 여기서는 PostgreSQL에 벡터 검색 기능(pgvector 계열로 추정)을 붙여 썼다.

TP/FP/FN/TN: 실제 취약한 것을 취약하다고 맞힘(TP), 아닌데 취약하다고 함(FP), 취약한데 놓침(FN), 아닌 것을 아니라고 맞힘(TN). 여기서는 CoT 설명까지 맞아야 정답으로 친다.

temperature, top-p: LLM 출력의 무작위성 설정. temperature가 낮을수록 같은 답을 반복하고, top-p는 확률 상위 몇 %의 단어 후보만 고를지 정한다.

VSP: 비교 대상 선행 연구. 정답 CWE 유형과 단순한 CWE 예시 하나를 준다. Baseline은 예시 없이 CWE 유형만 준다.

train/validation/test 분할: 학습용 70%, 조정용 15%, 최종 평가용 15%. 평가 데이터는 학습에 쓰지 않는다.

F1 점수: 정밀도(precision, 취약하다고 한 것 중 진짜 비율)와 재현율(recall, 진짜 취약 중 잡아낸 비율)의 조화평균. 둘 다 높아야 높다.

CWE 묶음 4개: C1 산술·형 변환 오류, C2 동시성(race condition, 잠금 오류), C3 메모리 안전(UAF, 버퍼 넘침), C4 입력 검증·논리·자원 처리.

race condition(경쟁 상태): 두 스레드가 같은 자원을 순서 보장 없이 건드려 결과가 실행 순서에 따라 달라지는 버그. 커널 드라이버에서 흔하다.

표 VII 읽는 법: 위에서부터 취약 여부만 판단, CoT까지 판단, 패치 정확도의 세 하위 표. 열은 모델, 행은 C1~C4.

incremental fine-tuning: 새 CVE가 나올 때마다 기존 모델에 이어서 추가 학습. 비용이 CVE 수에 비례해 선형으로 는다.

non-incremental fine-tuning: 매번 처음 모델에서 지금까지의 CVE 전체로 다시 학습. 비용이 제곱으로 는다.

관련 연구: GRACE는 코드 그래프 구조를 LLM에 넣어 취약 위치를 찾고, Vul-RAG는 지식 수준 RAG를 쓴다. 둘 다 CWE 유형 중심이라 CVE 단위 매칭은 아니다.

APPATCH: VSP 방식을 패치에 적용한 연구. 취약한 줄 위치를 입력으로 줘야 한다는 점이 AutoPatch와 다르다.

ThinkRepair: CoT 추론이 달린 버그·수정 코드 풀을 만들고 few-shot으로 수리. AutoPatch는 여기에 변수·함수 매핑을 더했다.

부록 A 시연: RGB 이미지를 받아 플러그인 필터(.so)를 동적으로 불러 처리하는 데몬을 직접 만들어 시험했다. → 카메라 영상 처리 파이프라인과 비슷한 구조라 감 잡기에 좋다(추정).

CVE-2024-27530: wasm3(WebAssembly 인터프리터)에서 모듈을 해제하면서 전역 목록에서 빼지 않아 생긴 UAF. 다른 프로그램의 CVE로 새 코드의 같은 패턴을 찾은 사례다.

그림 8 읽는 법: 왼쪽 위 LLM 생성 코드, 오른쪽 위 닮은 CVE 코드(wasm3), 아래가 패치 결과. +로 표시된 줄이 추가된 줄이다.

Listing 1: 의미 분석용 프롬프트. 일부러 보안 지식이 없는 엔지니어 역할을 주고 기능만 이름 없이 요약하게 한다. 보안 판단이 섞이면 비교가 흐려지기 때문이다.

Listing 4: 검증 프롬프트. RAG DB의 익명 변수·함수 설명과 매핑을 주고, 결과를 result, cot, root_cause JSON으로 받는다.

원문 1쪽

Large Language Models (LLMs)
① **LLM(대형 언어 모델):** 대량의 글과 코드를 학습해 다음 단어(토큰)를 예측하는 모델. ChatGPT, Code Llama, DeepSeek 등. 코드 생성에도 쓴다.

cutoff date
② **knowledge cutoff(학습 마감일):** 학습 데이터를 모은 마지막 시점. 그 뒤에 나온 CVE와 패치는 모델이 모른다. 이 논문의 문제의식 전체가 여기서 나온다.

newly disclosed CVEs
③ **CVE:** 공개된 개별 취약점에 붙는 고유 번호(예: CVE-2025-21671). MITRE가 관리한다. 특정 소프트웨어의 특정 버그 하나를 가리킨다.

tuning LLMs
④ **fine-tuning(미세조정):** 이미 학습된 모델을 새 데이터로 추가 학습해 가중치를 바꾸는 것. GPU 시간이 많이 들어서 CVE가 나올 때마다 하기 어렵다.

CWE examples
⑤ **CWE:** 취약점의 종류(유형) 분류. 예: CWE-416 Use After Free, CWE-787 버퍼 넘침. CVE는 사례, CWE는 범주다. 기존 연구는 교과서식 CWE 예제만 써서 실제 코드에 약하다.

multi-agent framework
⑥ **multi-agent:** 역할이 다른 LLM 호출(에이전트) 여러 개를 이어 붙여 일을 나눠 맡기는 구조. 여기서는 유사도 분석, 검증, 패치 세 에이전트.

Augmented Generation (RAG)
⑦ **RAG(검색 증강 생성):** 질문할 때 외부 DB에서 관련 자료를 찾아 프롬프트에 붙여 주는 방식. 모델 가중치를 바꾸지 않고 새 지식을 넣을 수 있다.

derived from 75 high-severity CVEs
⑧ **데이터셋 규모:** 고위험 CVE 75개에서 코드 조각 525개를 만들었다. 리눅스 커널 57개, Chromium 10개.

Chain-of-Thought (CoT)
⑨ **CoT(사고 사슬):** LLM에게 답만 내지 말고 단계별 추론을 쓰게 하는 프롬프트 기법. 복잡한 판단의 정확도가 오른다.

achieves 91.8% accuracy in CVE matching
⑩ **핵심 결과 1:** 생성된 코드에 맞는 CVE를 고르는 정확도 91.8%.

attains an F1-score of 90.3%
⑪ **핵심 결과 2:** 검증 F1 90.3%, 패치 정확도 94.1%, 미세조정 대비 50배 이상 저렴. F1은 5장 메모 참고.

roughly 30% of LLM-generated code suggestions
⑫ **문제의 크기:** 선행 연구에서 LLM 코드 제안의 약 30%가 이미 알려진 취약점을 포함했다.

explicit bug loca-
⑬ **기존 연구 한계 2:** 버그 위치(몇 번째 줄)를 사람이 알려 줘야 한다. 실제 개발에서는 위치를 모르는 상태에서 찾아야 한다.

원문 2쪽

security plugin for LLM-integrated IDEs
① **적용 범위:** IDE(VS Code 등) 보안 플러그인으로 LLM이 막 생성한 코드 조각을 검사한다. 이미 있는 대형 코드베이스 전체 감사나 펌웨어 백포트가 목적이 아니다.

three specialized LLM agents
② **세 에이전트:** 1) 닮은 CVE 찾기 2) 정말 취약한지 검증 3) 패치 생성. 패치 결과는 다시 2)로 돌려 검사한다.

calculating a semantic similarity score
③ **semantic similarity(의미 유사도):** 코드가 무엇을 하려는지(설명문, 키워드)가 CVE 코드와 얼마나 닮았는지 점수로 낸 것.

data flow analysis
④ **data flow analysis(데이터 흐름 분석):** 변수 값이 어디서 만들어져 어느 함수로 흘러가는지 추적하는 정적 분석. 이름이 달라도 구조가 같으면 잡아낸다.

pairwise ranking loss
⑤ **pairwise ranking loss:** 정답 CVE 점수가 오답 CVE 점수보다 일정 간격 이상 높도록 학습시키는 손실 함수. 분류가 아니라 순위를 맞추는 학습이다.

one-shot veri
⑥ **one-shot 예시:** 프롬프트에 풀이 예제를 하나 붙여 주는 것. 여기서는 찾아낸 CVE의 실제 취약 코드와 원인 분석이 예제가 된다.

525 code
⑦ **기여 3:** 최신 고위험 CVE 75개 기반 벤치마크 525개를 새로 만들었다. 공개 저장소 github.com/ai-llm-research/autopatch.

AutoPatch achieves a 5,230% lower cost
⑧ **비용:** 전체 데이터로 반복 미세조정하는 방식 대비 5,230% 저렴(약 53배). 11쪽 그림 7에서 계산 근거가 나온다.

TABLE I: Knowledge cutoff of code generation models
⑨ **표 I:** 대표 코드 생성 모델들의 학습 마감일. GPT-4o, o3-mini가 2023년 10월이다. 그 뒤 CVE는 모른다는 근거.

resource-intensive
⑩ **RAG를 쓰는 이유:** 미세조정 대신 외부 DB만 갱신하면 된다. 새 CVE가 나오면 DB 항목만 추가한다.

during inference
⑪ **inference(추론):** 학습이 끝난 모델을 실제로 호출해 답을 받는 단계. RAG 검색은 이 시점에 일어난다.

원문 3쪽

Fig. 2: Motivating examples using GPT-4o-2024-11-20.
① **그림 2 읽는 법:** 위는 cutoff 이후 CVE(취약하게 생성), 아래는 cutoff 이전 CVE(안전하게 생성). 같은 모델, 같은 방식의 프롬프트로 비교한 것이다.

75,314 commits
② **변화 속도:** 2024년 한 해 리눅스 커밋 7만 5천 건, 커널 관련 CVE 8,093건. 고정된 LLM 지식과 계속 바뀌는 코드 사이의 틈이 크다.

8,093 new Linux-kernel-related CVEs
③ **→ 임베디드 리눅스(EN683 BSP 포함)도 같은 커널 CVE 흐름을 따라가야 한다.** 카메라 펌웨어에 커널 패치를 백포트하는 일이 여기서 연결된다(추정).

zram device
④ **zram:** RAM 일부를 압축 블록 장치로 쓰는 리눅스 커널 기능. 메모리가 작은 임베디드 기기에서 스왑 대신 자주 쓴다.

dangling pointer
⑤ **dangling pointer(허상 포인터):** 메모리를 해제한 뒤에도 그 주소를 계속 가리키는 포인터. NULL로 지워 두지 않으면 나중에 다시 쓰일 수 있다.

use-after-free, CVE-2025-
⑥ **Use After Free(UAF, CWE-416):** 해제된 메모리를 다시 접근하는 버그. 공격자가 그 자리에 원하는 데이터를 넣으면 코드 실행까지 이어진다.

unable to avoid the
⑦ **동기 실험 1:** GPT-4o는 2025년에 공개된 CVE-2025-21671을 모르므로 같은 취약 패턴을 그대로 재현했다.

Secure Code Generation.
⑧ **동기 실험 2:** cutoff 이전에 공개된 CVE-2022-0185 사례는 GPT-4o가 패치된 형태로 안전하게 생성했다. 학습 데이터에 패치가 있었기 때문으로 본다.

원문 4쪽

Fig. 3: The overall architecture of AutoPatch.
① **그림 3 읽는 법:** 왼쪽에서 개발자 요청이 들어오고, 생성 코드가 Similarity Analyzer **→ Verifier → Patcher를 거친다.** 오른쪽 RAG DB가 각 단계에 CVE 정보를 공급한다.

CVE-2022-0185
② **CVE-2022-0185:** 리눅스 파일시스템 마운트 파라미터 처리에서 길이 계산 정수 오류로 힙 버퍼가 넘치는 취약점. 컨테이너 탈출에 실제 악용되었다.

A. AutoPatch Deployment Scenario
③ **배포 시나리오:** 개발자가 Copilot, Cursor 같은 LLM IDE를 쓰고, AutoPatch가 생성 코드를 IDE에 넣기 전에 가로채 검사한다.

AutoPatch intercepts it
④ **동작 위치:** 생성 직후, 코드베이스에 들어가기 전. 취약하지 않으면 그대로 통과, 취약하면 패치한 뒤 기능이 유지되는지 확인하고 넘긴다.

(i)
⑤ **두 능력:** (i) 의미 분석은 무엇을 하는 코드인지 비교, (ii) 데이터 흐름 분석은 변수·함수 이름을 지우고 역할과 흐름만 비교. 둘을 합쳐야 구조가 다르지만 같은 버그인 경우도 잡는다.

Unified Similarity Model
⑥ **Unified Similarity Model:** 키워드, 설명문, 변수, 함수 네 가지 유사도에 가중치를 곱해 더한 점수 모델. 가중치를 학습으로 정한다.

원문 5쪽

top 10,000 most frequently used tags from Stack Over-
① **키워드 사전:** Stack Overflow 상위 태그 1만 개를 키워드 후보로 쓴다. 코드 설명에서 이 태그에 해당하는 단어만 뽑는다.

Jaccard similarity score
② **Jaccard 유사도:** 두 집합의 교집합 크기 ÷ 합집합 크기. 키워드가 많이 겹칠수록 1에 가깝다.

rapidfuzz
③ **rapidfuzz:** 철자가 조금 달라도 비슷한 문자열을 찾는 퍼지 매칭 라이브러리. 유사도 80% 이상이면 같은 키워드로 친다(예: alloc과 allocate).

high-dimensional vector representations
④ **embedding(임베딩):** 문장이나 코드 설명을 수백~수천 차원 숫자 벡터로 바꾼 것. 뜻이 비슷하면 벡터 방향도 비슷해진다.

and cosine similarity is employed
⑤ **코사인 유사도:** 두 벡터 사이 각도의 코사인. 방향이 같으면 1, 직각이면 0. 길이가 아닌 방향(의미)만 비교한다.

Data Flow Graph (DFG)
⑥ **DFG(데이터 흐름 그래프):** 노드는 변수·함수, 화살표는 값이 흘러가는 방향. 그림 4가 zram 코드에서 뽑은 DFG다.

CodeQL
⑦ **CodeQL:** GitHub의 정적 분석 도구. 소스 코드를 DB로 바꾼 뒤 SQL 비슷한 질의로 데이터 흐름을 찾는다. **→ 펌웨어 C 코드 감사에도 그대로 쓸 수 있는 도구다.**

removing literal
⑧ **핵심 아이디어:** 변수·함수 이름을 지우고 역할 설명(symbolic description)으로 바꾼다. 이름을 바꿔도 같은 취약 패턴이면 매칭된다.

we again employ cosine similarity
⑨ 변수 설명끼리, 함수 설명끼리 각각 코사인 유사도를 낸다. 결과적으로 점수는 키워드·설명·변수·함수 4개.

원문 6쪽

Fig. 5: The target code refers
① **그림 5 읽는 법:** LLM에게 DFG 정보를 주고 각 변수·함수를 이름 없이 설명하게 한 결과. 예: vzalloc **→ 0으로 초기화한 연속 가상 메모리 할당.**

Then, the score S for a given
② **식 (2) 통합 점수:** 네 유사도의 가중합. 가중치 w1~w4는 데이터로 학습한다. 점수가 가장 높은 CVE를 고른다.

margin hyperparameter
③ **margin(여유폭) m:** 정답 점수가 오답 점수보다 최소 m만큼은 높아야 손실이 0이 된다. 하이퍼파라미터는 학습 전에 사람이 정하는 값.

(i.e., Adam)
④ **Adam:** 가장 널리 쓰는 경사하강 최적화 알고리즘. 손실이 줄어드는 방향으로 가중치를 조금씩 조정한다.

avoid unnecessary code
⑤ **검증 단계가 따로 있는 이유:** 닮은 CVE를 찾았다고 해서 정말 취약한 것은 아니다. 취약하지 않은 코드를 괜히 고치지 않게 한다.

Automated Program
⑥ APR(Automated Program Repair, 자동 프로그램 수리): 버그를 자동으로 고치는 연구 분야. 패치 **→ 검증 → 다시 패치 루프가 표준 구조다.**

원문 7쪽

Fig. 6:
① **그림 6 읽는 법:** 점선 상자와 파란 칸이 검증용, 초록 칸이 패치용 프롬프트 부품. system **→ one-shot 예시 → user 순서로 이어 붙인다.**

role-based structure
② **role-based 프롬프트:** system(규칙과 역할), user(질문), assistant(답) 역할로 나눠 쓰는 대화형 LLM 입력 형식.

"variable_1" and "function_1"
③ 이름을 variable_1, function_1로 추상화한다. 그래야 다른 이름의 코드에도 같은 CVE 설명이 적용된다.

in-context demonstration
④ **in-context learning:** 가중치를 바꾸지 않고 프롬프트 안의 예시만 보고 따라 하게 하는 것. one-shot 예시가 그 역할을 한다.

one-hop calling functions
⑤ **one-hop calling function:** 대상 함수가 직접 호출하는 함수(한 단계 거리). 그 정의까지 같이 줘야 LLM이 동작을 이해한다.

boolean verdict
⑥ **검증 출력:** 취약 여부(true/false)와 근본 원인(root cause) 설명을 JSON으로 받는다.

(iii)
⑦ **Code Patcher 역할 세 가지:** 패치 예시 구성, 패치 프롬프트 생성, 결과를 다시 Verifier로 넘겨 재검증.

원문 8쪽

TABLE II: Trivial and non-trivial code augmentations used in
① **표 II 읽는 법:** trivial(겉모양만 바꿈: 공백, 주석, 도달 불가 코드)과 non-trivial(구조 변경: 이름 변경, 위험 함수 안전 사용)로 나눈 데이터 증강.

Feedback Loop.
② **패치 → 재검증 루프:** 취약점이 사라지거나 정해 둔 최대 횟수에 도달할 때까지 반복한다.

LangChain
③ **LangChain:** LLM 호출, 프롬프트 템플릿, DB 검색을 묶어 에이전트를 만드는 파이썬 프레임워크.

with vector search
④ **vector DB(벡터 검색):** 임베딩 벡터를 저장해 두고 가장 가까운 벡터를 빠르게 찾는 DB. 여기서는 PostgreSQL에 벡터 검색 기능(pgvector 계열로 추정)을 붙여 썼다.

We develop a cus-
⑤ **데이터 수집:** GitHub Advisory DB, Openwall, Chromium 이슈 트래커를 크롤링해 2024년 말~2025년 고위험 CVE 75개를 모았다.

this yields 375 code snippets
⑥ **구성:** CVE 75개 × 생성 모델 5개 = 375개, 여기에 증강 코드 150개(취약 75, 패치 75)를 더해 525개.

Non-trivial augmentations more meaningfully modify code
⑦ **증강 목적:** 변수명을 exploitFlag처럼 바꾸거나 strcpy를 안전하게 쓰게 해서, 키워드만 보고 판단하는 모델을 걸러 낸다. 겉모습에 속지 않는지 시험하는 것.

원문 9쪽

True Positive (TP)
① **TP/FP/FN/TN:** 실제 취약한 것을 취약하다고 맞힘(TP), 아닌데 취약하다고 함(FP), 취약한데 놓침(FN), 아닌 것을 아니라고 맞힘(TN). 여기서는 CoT 설명까지 맞아야 정답으로 친다.

temperature of 0.2 and a top-p value of 0.9
② **temperature, top-p:** LLM 출력의 무작위성 설정. temperature가 낮을수록 같은 답을 반복하고, top-p는 확률 상위 몇 %의 단어 후보만 고를지 정한다.

VSP [13]
③ **VSP:** 비교 대상 선행 연구. 정답 CWE 유형과 단순한 CWE 예시 하나를 준다. Baseline은 예시 없이 CWE 유형만 준다.

we manually verify
④ 정답 라벨(ground truth)을 사람이 직접 달았다. 데이터가 작고 일부 수작업이라는 점은 6장 한계에서도 인정한다.

70:15:15
⑤ **train/validation/test 분할:** 학습용 70%, 조정용 15%, 최종 평가용 15%. 평가 데이터는 학습에 쓰지 않는다.

accuracy in mapping each code snippet
⑥ **통합 모델 결과:** 테스트 세트에서 CVE 매칭 정확도 91.78%.

2:1
⑦ **클래스 불균형 보정:** 취약 코드가 훨씬 많아서 CVE마다 취약:비취약 = 2:1로 샘플링했다. 실제 개발 현장의 비율과는 다를 수 있다(추정).

92.00% F1-score
⑧ **F1 점수:** 정밀도(precision, 취약하다고 한 것 중 진짜 비율)와 재현율(recall, 진짜 취약 중 잡아낸 비율)의 조화평균. 둘 다 높아야 높다.

원문 10쪽

and 90.32% F1-score
① 취약 여부 + CoT 설명까지 모두 맞아야 정답으로 칠 때도 GPT-4o 정확도 88.29%, F1 90.32%.

the highest patching accuracy at 94.12%
② **패치 정확도:** GPT-4o 94.12%, o3-mini 87.07%, DeepSeek-R1 85.04%.

55.56% and
③ **비교:** VSP 55.56%, baseline 46.38%. 단순 CWE 예시로는 실제 CVE 패치가 잘 안 된다는 근거.

lowest average loop count (1.08)
④ **루프 횟수:** GPT-4o는 평균 1.08회. 대부분 첫 패치로 검증을 통과한다. 최대는 o3-mini 10회.

C1 (Arithmetic & Type Errors), C2 (Con-
⑤ **CWE 묶음 4개:** C1 산술·형 변환 오류, C2 동시성(race condition, 잠금 오류), C3 메모리 안전(UAF, 버퍼 넘침), C4 입력 검증·논리·자원 처리.

and C3, with VSP achieving only 31.3% and 27.8%
⑥ **기존 방식이 가장 약한 곳:** 동시성(C2)과 메모리 안전(C3). 맥락 없이는 잡기 어려운 유형이다.

race conditions and improper lock-
⑦ **race condition(경쟁 상태):** 두 스레드가 같은 자원을 순서 보장 없이 건드려 결과가 실행 순서에 따라 달라지는 버그. 커널 드라이버에서 흔하다.

원문 11쪽

TABLE VII: Comparison of Performance Based on CWE Type.
① **표 VII 읽는 법:** 위에서부터 취약 여부만 판단, CoT까지 판단, 패치 정확도의 세 하위 표. 열은 모델, 행은 C1~C4.

incremental fine-tuning
② **incremental fine-tuning:** 새 CVE가 나올 때마다 기존 모델에 이어서 추가 학습. 비용이 CVE 수에 비례해 선형으로 는다.

non-incremental fine-tuning
③ **non-incremental fine-tuning:** 매번 처음 모델에서 지금까지의 CVE 전체로 다시 학습. 비용이 제곱으로 는다.

peaking at only $5.7
④ **AutoPatch 비용:** CVE 75개 처리 시 최대 5.7달러로 거의 일정. 미세조정은 37.3~303.8달러.

tuning at 5-CVE intervals
⑤ 5,230%는 303.8 ÷ 5.7 ≈ 53배에서 나온 수치. 비용은 OpenAI API 가격표 기준 추정이고 GPU 직접 운용 비용은 아니다.

fundamentally
⑥ **근본 한계:** DB에 선례가 있는 알려진 취약점만 찾는다. 제로데이나 새로운 공격 패턴은 못 잡는다.

readily extendable to existing codebases
⑦ **저자 주장:** 코드 출처에 의존하지 않으므로 기존 코드베이스에도 확장 가능하다. 다만 이 논문에서 기존 코드베이스로 실험하지는 않았다. → **카메라 커널 코드에도 쓸 수 있을지는 확인되지 않았다(추정).**

원문 12쪽

relatively small number of Chrome-related CVEs
① **한계 2:** Chrome은 고위험 취약점을 패치 배포 후 일정 기간 공개하지 않아 CVE가 10개뿐이다.

GRACE [43]
② **관련 연구:** GRACE는 코드 그래프 구조를 LLM에 넣어 취약 위치를 찾고, Vul-RAG는 지식 수준 RAG를 쓴다. 둘 다 CWE 유형 중심이라 CVE 단위 매칭은 아니다.

APPATCH proposed an LLM-based automated patching
③ **APPATCH:** VSP 방식을 패치에 적용한 연구. 취약한 줄 위치를 입력으로 줘야 한다는 점이 AutoPatch와 다르다.

ThinkRepair
④ **ThinkRepair:** CoT 추론이 달린 버그·수정 코드 풀을 만들고 few-shot으로 수리. AutoPatch는 여기에 변수·함수 매핑을 더했다.

achieving an F1-
⑤ **결론 요약:** GPT-4o 기준 검증 F1 90.3%, 패치 94.1%, 동시성·검증 로직 유형에서 특히 강하다.

원문 14쪽

Image-Processing
① **부록 A 시연:** RGB 이미지를 받아 플러그인 필터(.so)를 동적으로 불러 처리하는 데몬을 직접 만들어 시험했다. → **카메라 영상 처리 파이프라인과 비슷한 구조라 감 잡기에 좋다(추정).**

CVE-2024-27530
② **CVE-2024-27530:** wasm3(WebAssembly 인터프리터)에서 모듈을 해제하면서 전역 목록에서 빼지 않아 생긴 UAF. 다른 프로그램의 CVE로 새 코드의 같은 패턴을 찾은 사례다.

Variables: plg
③ **매핑 예:** 새 코드의 plg ↔ CVE 코드의 module, g_plugins ↔ runtime. 이름이 전혀 달라도 역할로 짝을 맞췄다.

원문 15쪽

Fig. 8: AutoPatch with load_plugin function of Image-Processing Daemon.
① **그림 8 읽는 법:** 왼쪽 위 LLM 생성 코드, 오른쪽 위 닮은 CVE 코드(wasm3), 아래가 패치 결과. +로 표시된 줄이 추가된 줄이다.

g_plugins[index] = NULL; /* remove from registry */
② **패치 핵심:** 초기화 실패로 플러그인을 해제할 때 전역 목록 칸도 NULL로 지운다. 해제된 객체를 목록이 계속 가리키는 UAF를 막는다.

원문 16쪽

Listing 1: Prompt used for semantic analysis.
① **Listing 1:** 의미 분석용 프롬프트. 일부러 보안 지식이 없는 엔지니어 역할을 주고 기능만 이름 없이 요약하게 한다. 보안 판단이 섞이면 비교가 흐려지기 때문이다.

원문 17쪽

Listing 4:

① **Listing 4:** 검증 프롬프트. RAG DB의 익명 변수·함수 설명과 매핑을 주고, 결과를 result, cot, root_cause JSON으로 받는다.

원문 19쪽

Fig. 9:
① **그림 9:** 동기 예시(zram) 실제 프롬프트 전체. 패치 결과는 vfree 뒤에 zram->table = NULL 한 줄 추가. 프롬프트 설계를 따라 해 보려면 이 그림을 그대로 참고하면 된다.